Data security

Banking training API

KENNE NZANGEM Lionel, MELI Junior & SONDEJI Karl

1. Introduction et contexte

1.1. Introduction

Ce rapport documente le travail réalisé sur une mini-application PHP pour entester la sécurité. L'objectif principal était de comprendre les mécanismes des vulnérabilités web courantes (OWASP Top 10) en les implémentant volontairement, pour ensuite appliquer les correctifs de sécurité appropriés.

1.2. Contexte

Le projet s'articule autour d'une simulation bancaire simplifiée ("BankingTraining") permettant la gestion d'utilisateurs et de comptes, développée en PHP, servie par Apache et utilisant MySQL. Le travail a été divisé en plusieurs phases : conception d'une base de données avec rôles et relations, création d'une version vulnérable exploitable (injections SQL et abus de privilèges), correction par requêtes préparées et validation des entrées, renforcement de l'authentification par hachage bcrypt, puis protection des échanges par certificat X.509 et HTTPS avec redirection et cookies sécurisés, enfin, journalisation des connexions ainsi que des actions sensibles pour la détection d'incidents.

2. Conception de la base de données

2.1 Schéma de la base de données

La base de données nommée 'bankingtraining' contient 3 tables et suit le schéma étoile avec 2 tables de tables de dimensions et 1 table de faits, 'dim_user' pour gérer les différents comptes utilisateurs (des rôles ont été définis, tel que utilisateur, analyst ou encore admin), 'dim_account' pour gérer les différents comptes bancaire (épargne, business etc.) et la table 'fact_transactions' qui a pour clé étrangère les clés primaires des tables précédentes et qui regroupe les différentes transactions bancaires.

2.2 Contraintes de la base de données

Les différentes clés primaires et étrangères ont pour contraintes d'être des champs non nuls et des identifiants uniques.

```sql
-- Table structure for table `dim_user`
--

DROP TABLE IF EXISTS `dim_user`;
CREATE TABLE IF NOT EXISTS `dim_user` (
  `user_id` int(11) NOT NULL AUTO_INCREMENT,
  `username` varchar(200) NOT NULL,
  `password` varchar(100) NOT NULL,
  `full_name` varchar(100) NOT NULL,
  `user_role` varchar(50) NOT NULL,
  `country` varchar(50) NOT NULL,
  `signup_date` date NOT NULL,
  PRIMARY KEY (`user_id`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1 AUTO_INCREMENT=10;

--
```

Pour générer de manière aléatoire des valeurs de départ pour notre base de données, nous avons utilisé le package `Faker`de python.

```python
from faker import Faker
import random
from datetime import date

fake = Faker("fr_FR")

roles = ["Customer", "Customer", "Customer", "Analyst"]
countries = ["FR", "BE", "CH", "CA"]

n_users = 50
start_id = 5

rows = []

for i in range(n_users):

    user_id = start_id + i
    full_name = fake.name()
    username = full_name.lower().replace(" ", ".")
    password = fake.password(length=6)
    role = random.choice(roles)
```

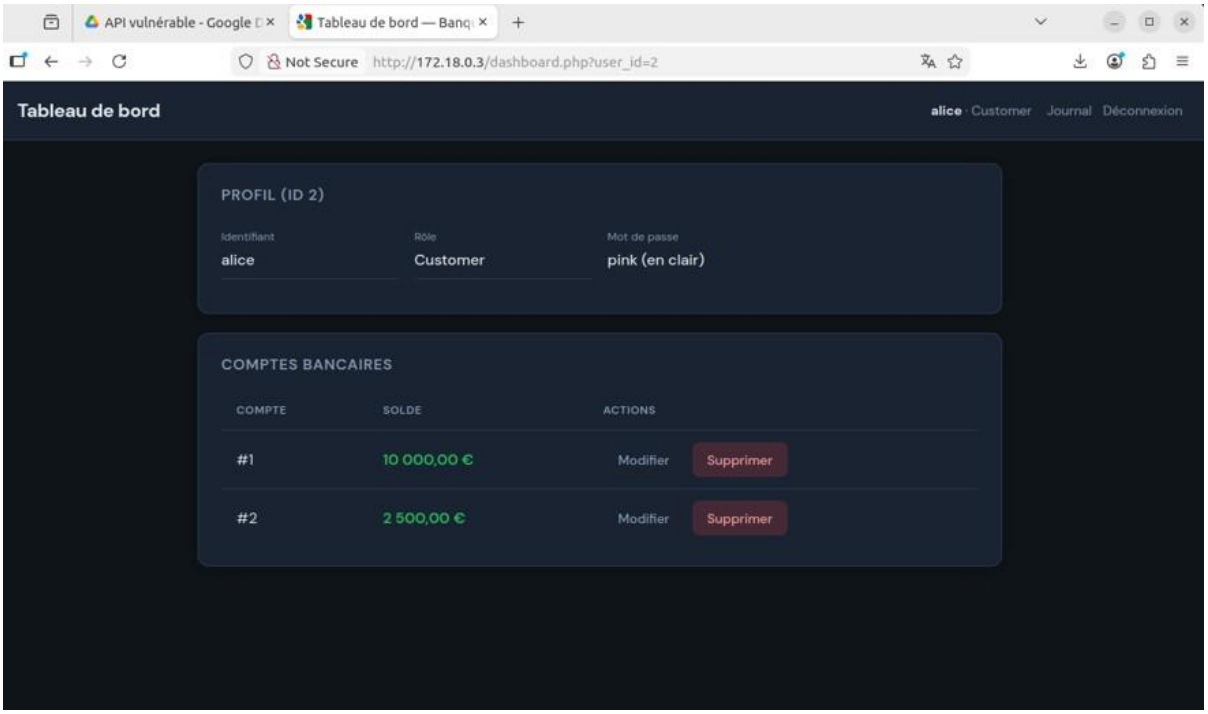
```python
for i in range(n_users):

    (variable) full_name: str
    full_name = fake.name()
    username = full_name.lower().replace(" ", ".")
    password = fake.password(length=6)
    role = random.choice(roles)
    country = random.choice(countries)
    signup_date = fake.date_between(start_date="-2y", end_date="today")

    row = f"({user_id}, '{username}', '{password}', '{full_name}', '{role}', '{country}', '{signup_date}')"

    rows.append(row)

print("INSERT INTO dim_user")
print("(`user_id`,`username`,`password`,`full_name`,`user_role`,`country`,`signup_date`)")
print("VALUES")

print(",\n".join(rows) + ";")
```

3. Création d'une première application vulnérable

3.1 Points d'entrée vulnérables

L'application initiale a été conçue sans aucune mesure de sécurité avec cependant plusieurs exigences fonctionnelles, possibilité d'inscription, d'authentification, de connexion, CRUD sur les comptes, interface web, exposant directement les failles. Voici la structure des fichiers clés :

- config.php : Établit une connexion non sécurisée à la base MySQL.
- login.php : Script d'authentification vulnérable aux injections SQL les entrées utilisateur (`$_GET`, `$_POST`) sont insérées directement dans les requêtes sans requêtes préparées.
- dashboard.php : Tableau de bord utilisateur présentant des failles de contrôle d'accès (IDOR).
- Mots de passe : stockés et lus tels quels dans la colonne `password`.
- http : pas de HTTPS ; trafic et identifiants transitent en clair.



Des points d'entrée permettent en outre des abus de privilèges (accès au tableau de bord d'un autre utilisateur via `dashboard.php?user_id=X), la modification/suppression de comptes sans vérification d'appartenance (`edit_account.php`, `delete_account.php`), l'élévation de privilèges via `make_admin.php`. Cette version est donc exploitable pour des injections SQL et des contournements de contrôle d'accès.

3.2 Identification et exploitations des injections SQL

Les champs identifiant et mot de passe utilisés dans une requête du type
`SELECT * FROM users WHERE username = '...' AND password = '...'`.
Par conséquent cela permet plusieurs scénarios.

1. Contournement d'authentification
   - Identifiant `alice'#`, mot de passe quelconque : le `#` commente le reste de la requête, qui devient `WHERE username = 'alice'` → une seule ligne renvoyée → connexion en tant qu'alice.
   - Variante : mot de passe `' OR username='alice' AND '1'='1` pour restreindre à la ligne d'alice.

2. Accès aux données d'autres utilisateurs
   - Saisie manuelle de `dashboard.php?user_id=2` (ou autre ID) pour afficher le tableau de bord d'un autre utilisateur.

3. Modification / suppression de données d'autrui
   - Modification de l'URL ou de l'`id` dans les formulaires pour cibler un compte dont le `user_id` ne correspond pas à l'utilisateur connecté et ainsi modifier le montant de ses comptes voir même supprimer un compte qui ne nous appartient pas.

4. Actions réservées à l'admin
   - Accès à `make_admin.php`. Si un utilisateur, disons l'utilisateur 3 se connecte, il suffit d'ajouter '& admin' dans l'url de connexion à côter de 'user_id=3' pour avoir accès aux droits d'administrateur.

Chaque scénario est documenté dans le README (paramètre utilisé, effet sur la requête ou l'accès, impact sur la confidentialité, l'intégrité ou la disponibilité des données).

3.3 Sécurisation face aux injections SQL et validations d'entrées

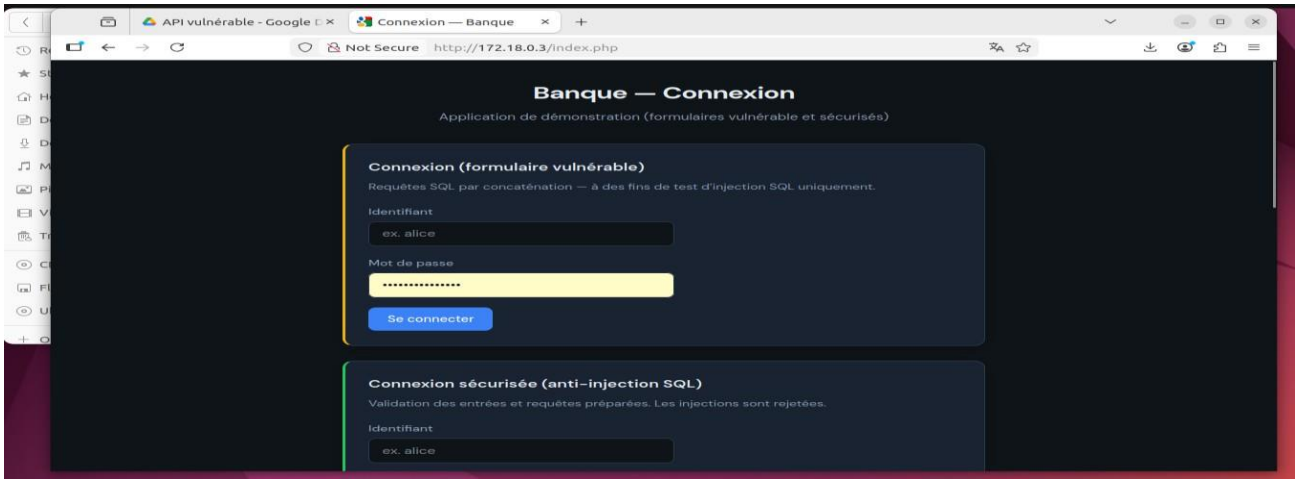
Pour remédier au problème des injections SQL, nous avons paramétré les requêtes en faisant en sorte que l'authentification et les opérations sensibles en base passent par des requêtes préparées, utilisation de `mysqli_prepare`et `mysqli_stmt_bind_param`. Résultat, les entrées sont liées en paramètres et ne sont jamais concaténées au SQL. Les payloads d'injection sont traités comme des chaînes littérales et n'altèrent plus la requête.

```php
// --- Authentification par requête préparée (plus d'injection SQL possible) ---

$stmt = mysqli_prepare($conn, "SELECT id, username, role FROM users WHERE username = ? AND password = ?");
if (!$stmt) {
    $_SESSION['secure_validation_errors'] = ["Erreur technique. Veuillez réessayer."];
    header("Location: index.php#connexion-securisee");
    exit;
}

mysqli_stmt_bind_param($stmt, "ss", $username, $password);
mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);
mysqli_stmt_close($stmt);

if ($result && mysqli_num_rows($result) === 1) {
    $user = mysqli_fetch_assoc($result);
    $_SESSION['user_id'] = $user['id'];
    $_SESSION['username'] = $user['username'];
    $_SESSION['role'] = $user['role'];
    $_SESSION['logged_via_secure'] = true;
```

De plus, nous avons ajouté des contraintes de validation côté serveur, en définissant le type, la longueur ainsi que le format attendu pour l'identifiant et le mot de passe. Par exemple pour l'identifiant nous avons utilisé le package Regex pour empêcher les caractères non-souhaités (dont les guillemets souvent utilisés pour des injections), et pour le mot de passe, nous avons imposé des contraintes de longueur ainsi que des caractères spéciaux pour améliorer sa robustesse (plus de mot de passe du type 1234).

```php
$errors = [];

if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
    header("Location: index.php");
    exit;
}

$username = isset($_POST['username_secure']) ? trim((string) $_POST['username_secure']) :
$password = isset($_POST['password_secure']) ? (string) $_POST['password_secure'] : '';

// --- Validation des entrées côté serveur ---

// Type et présence
if ($username === '') {
    $errors[] = "L'identifiant est obligatoire.";
}
if ($password === '') {
    $errors[] = "Le mot de passe est obligatoire.";
}

// Longueur maximale
if (strlen($username) > USERNAME_MAX_LENGTH) {
    $errors[] = "L'identifiant ne doit pas dépasser " . USERNAME_MAX_LENGTH . " caractères
}
if (strlen($password) > PASSWORD_MAX_LENGTH) {
    $errors[] = "Le mot de passe ne doit pas dépasser " . PASSWORD_MAX_LENGTH . " caractèr
}

// Format attendu (caractères autorisés pour l'identifiant)
if ($username !== '' && !preg_match(USERNAME_FORMAT_REGEX, $username)) {
    $errors[] = "L'identifiant ne doit contenir que des lettres, chiffres, points, tirets
}

// Caractères spéciaux / caractères de contrôle dans le mot de passe (optionnel : rejeter
if (strpos($password, "\0") !== false) {
```

Enfin nous avons ajouté des contrôles d'accès ainsi que des vérifications. Le `user_id` dans l'URL est ignoré ; seul l'utilisateur connecté voit ses données. La modification/suppression de comptes vérifie que le compte appartient à l'utilisateur connecté ; sinon, redirection avec « Accès refusé ». Seul l'utilisateur `admin` peut accéder à `make_admin.php`. Les attaques de l'étape 2 ont été rejouées sur le formulaire « Connexion sécurisée », le bypass par commentaire ou par `OR` est rejeté, les accès aux données/comptes d'autrui et élévation de privilèges sont bloqués.



4. Protection des mots de passe et renforcement de l'authentification

Mise en place d'un stockage sécurisé des mots de passe à l'aide de l'algorithme de hachage `Bcrypt` adapté aux mots de passe (avec sel et paramètres de coût appropriés).

```
$stmt = mysqli_prepare($conn, "SELECT id, username, role, password_hash FROM users_hashed WHERE username = ?");
if (!$stmt) {
    $_SESSION['hashed_validation_errors'] = ["Erreur technique. Veuillez réessayer."];
    header("Location: index.php#encryptage");
    exit;
}
mysqli_stmt_bind_param($stmt, "s", $username);
mysqli_stmt_execute($stmt);
$result = mysqli_stmt_get_result($stmt);
$user = mysqli_fetch_assoc($result);
mysqli_stmt_close($stmt);

if ($user && password_verify($password, $user['password_hash'])) {
    unset($user['password_hash']);
    $_SESSION['user_id'] = $user['id'];
    $_SESSION['username'] = $user['username'];
    $_SESSION['role'] = $user['role'];
    $_SESSION['logged_via_secure_hash'] = true;
    unset($_SESSION['hashed_validation_errors']);
    app_log('LOGIN_SUCCESS', 'Connexion réussie (formulaire haché)', ['username' => $user['username']]);
    header("Location: dashboard.php");
    exit;
}

app_log('LOGIN_FAILURE', 'Tentative de connexion échouée (formulaire haché)', ['username' => $username]);
$_SESSION['hashed_validation_errors'] = ["Identifiant ou mot de passe invalide."];
header("Location: index.php#encryptage");
exit;
```

Adaptation du processus d'inscription et de connexion utilisant le nouveau mécanisme de hachage, ajout de 3 nouveaux fichiers, `register_hashed.php` pour enregistrer le mot de passe avec un hachage lors de l'inscription, `login_secure_hash.php` pour vérifier le mot de passe d'un utilisateur déjà inscrit, `change_password_hashed.php` en cas de changement de mot de passe.

```php
$hash = password_hash($password, PASSWORD_BCRYPT, ['cost' => BCRYPT_COST]);
$stmt = mysqli_prepare($conn, "INSERT INTO users_hashed (username, password_hash, role) VALUES (?, ?, ?)");
mysqli_stmt_bind_param($stmt, "sss", $username, $hash, $role);
if (!mysqli_stmt_execute($stmt)) {
    $_SESSION['hashed_validation_errors'] = ["Erreur lors de l'inscription."];
    header("Location: index.php#encryptage");
    exit;
}
mysqli_stmt_close($stmt);
```

Les mots de passe ne sont jamais stockés ou affichés en texte clair, ni dans la base de données, ni dans les journaux (logs). Pour vérifier si un mot de passe en clair correspond bien au hachage stocké en base (celui produit par `password_hash()`), on utilise la fonction PHP `password_verify()`, qui retourne `true` si ça correspond, `false` sinon. Le hachage stocké est une chaîne contenant tout ce qu'il faut (l'algorithme utilisé, le coût, le sel et l'empreinte dérivée du mot de passe plus le sel) pour refaire le calcul à l'envers, non pas en déchiffrant le mot de passe, mais en lisant le mot de passe fournit et en lui appliquant le même processus d'encryptage, et si l'empreinte qui en résulte correspond à l'empreinte du hachage stocké alors les 2 mots de passe correspondent.

```
karl@karl-virtual-machine: ~/Downloads/Projet SQL injectio...
------+
karl@karl-virtual-machine:~/Downloads/Projet SQL injections banking_6/Projet SQL
injections banking$ docker compose exec db mysql -uroot -proot bankingtraining
-e "SELECT id, username, password_hash, role FROM users_hashed;"
WARN[0000] /home/karl/Downloads/Projet SQL injections banking_6/Projet SQL injec
tions banking/docker-compose.yml: the attribute `version` is obsolete, it will b
e ignored, please remove it to avoid potential confusion
mysql: [Warning] Using a password on the command line interface can be insecure.
----+----------+-------------------------------------------------------------+
----------+
 id | username | password_hash                                               |
role     |
----+----------+-------------------------------------------------------------+
----------+
  1 | karl     | $2y$12$rhljQFRhL86cs8zC3gqjj.NT1tqUBO5pqLFyMmc5jXjf4iq5fnAv. |
Admin    |
  2 | junior   | $2y$12$6.Ltk09XlKCwRAQ5hh5nou8BS/7lQ0sEH/ClIqGHJDgH40FUABIOy |
Analyst  |
  3 | Am◆lie   | $2y$12$bG83KnRPwX25SgHfPVQZjubQRYN1GJLhPhRNnjfYTVmyVlDnZ5bI2 |
Customer |
----+----------+-------------------------------------------------------------+
----------+
karl@karl-virtual-machine:~/Downloads/Projet SQL injections banking_6/Projet SQL
```

Cependant, l'une des limites à l'utilisation de Bcrypt pour encrypter les mots de passe, est le fait que l'utilisation du coût de 12, bien qu'augmentant la résistance aux attaques par force brute, celui-ci augmente le temps de calcul et donc la charge CPU. Un coût de 12 signifie que le nombre d'itérations qu'effectue l'algorithme sera de 4096, c'est à dire que celui-ci va recalculer un hash 4096 fois en exécutant un processus interne appelé Expensive Key Schedule qui consiste à mélanger profondément le mot de passe, le sel, et l'état interne du chiffrement (ici blowfish), ce qui par conséquent a un certain coût et qui intègre volontairement une certaine lenteur pour ralentir les attaquants.
L'autre limite est qu'en cas de fuite dans la base de données, si des utilisateurs sont supprimés puis réintégrés, on ne peut pas à partir du hash retrouver le mot de passe originel, c'est-à-dire qu'il n'y a pas de clé pour déchiffrer l'empreinte du mot de passe.

5. Certificat numérique, HTTPS et journalisation

5.1. Certificat numérique

La génération d'un certificat auto-signé repose sur l'utilisation de l'outil OpenSSL, qui permet de créer à la fois une clé privée et un certificat X.509 associé.
Dans un premier temps, on se place dans le répertoire dédié aux certificats, puis on exécute une commande qui demande à OpenSSL de générer une nouvelle paire clé/certificat. Cette commande utilise l'algorithme RSA avec une taille de clé de 2048 bits, ce qui constitue aujourd'hui un standard de sécurité courant. Le certificat est auto-signé grâce à l'option -x509, ce qui signifie qu'il est signé par sa propre clé privée et non par une autorité de certification externe.

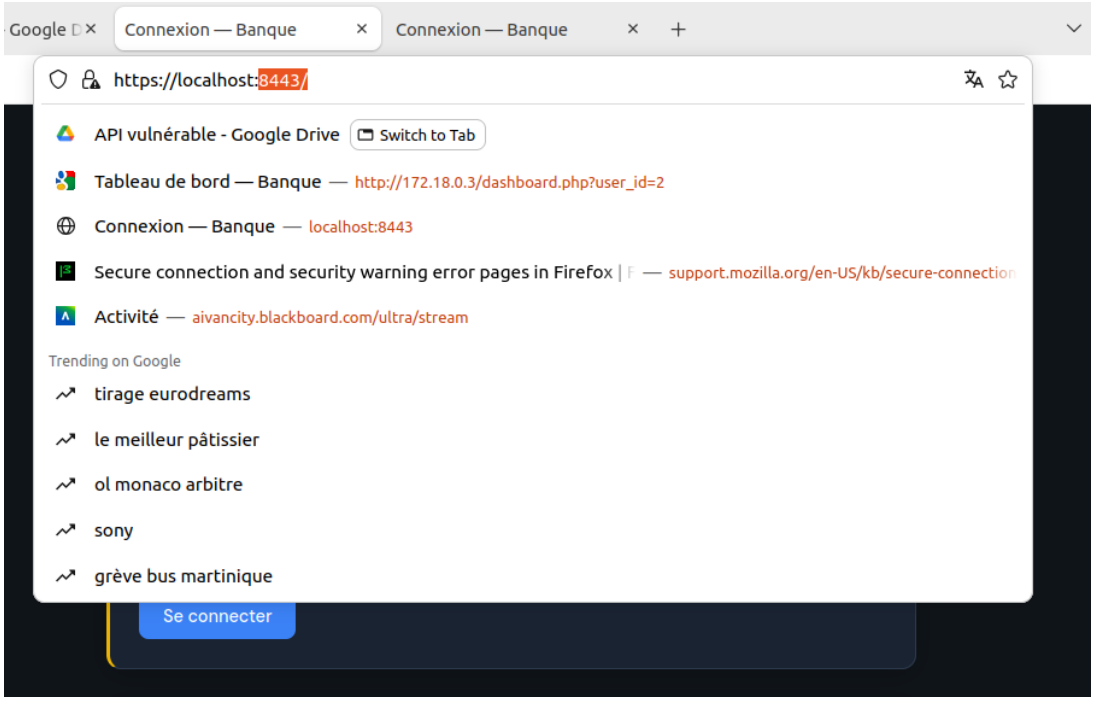
La durée de validité du certificat est fixée à 1 an (-days 365). L'option -nodes permet de ne pas chiffrer la clé privée avec un mot de passe, ce qui simplifie son utilisation dans des environnements de test ou de développement. Enfin, le champ -subj permet de définir directement les informations d'identité du certificat (comme le nom commun CN=localhost, l'organisation et le pays), sans interaction utilisateur.

Une fois le certificat généré, plusieurs vérifications permettent de s'assurer de sa validité et de sa cohérence. On effectue plusieurs vérifications, on vérifie sa validité temporelle et son contenu, sa conformité syntaxique et sa cohérence avec la correspondance clé/certificat, qui sera ensuite condensé à l'aide de la fonction de hachage MD5 afin d'obtenir des empreintes comparables, si les deux sont identiques alors la paire est cohérente.
Ces étapes sont essentielles pour éviter des erreurs de configuration, notamment lors de la mise en place de connexions sécurisées (HTTPS, TLS).

5.2. Serveur web HTTPS

Le serveur web utilisé est Apache HTTP Server, avec le module mod_ssl activé afin de gérer les communications chiffrées via TLS. Un VirtualHost est configuré pour écouter sur le port 443, qui est le port standard du protocole HTTPS. Dans ce contexte, le serveur utilise un certificat (server.crt) et une clé privée (server.key), montés en lecture seule depuis un répertoire dédié.

Afin d'éviter toute transmission de données sensibles en clair, un mécanisme de redirection est mis en place. Toute requête arrivant en HTTP (par exemple sur le port 8080) est automatiquement redirigée vers HTTPS avec un code de statut 301 (redirection permanente). Cela garantit que les formulaires de connexion ne sont jamais envoyés en clair et que les identifiants et mots de passe transitent uniquement via un canal chiffré.

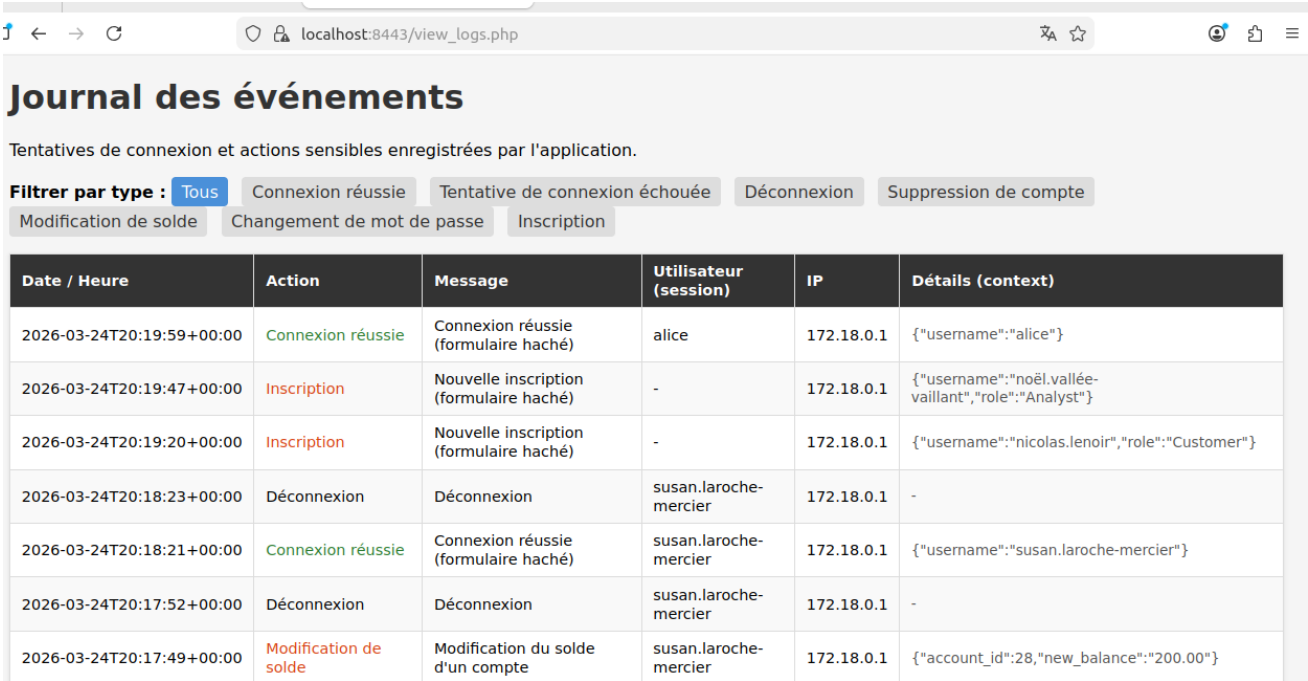


Le serveur envoie également un en-tête HTTP Strict-Transport-Security (HSTS), qui indique au navigateur qu'il doit toujours utiliser HTTPS pour communiquer avec ce site pendant une durée donnée (ici un an), cela permet d'éviter les attaques de type *downgrade* (forcer un retour en HTTP).

Les cookies de session sont configurés avec plusieurs attributs de sécurité, Secure (le cookie n'est envoyé que sur des connexions HTTPS), HttpOnly (le cookie n'est pas accessible via JavaScript, ce qui limite les risques en cas de faille XSS) et SameSite=Strict (le cookie n'est pas envoyé lors de requêtes cross-site, réduisant les risques d'attaques CSRF).
Le certificat utilisé dans le HTTPS joue un double rôle essentiel, car il permet l'authentification du serveur avec un client (navigateur) qui vérifie qu'il communique bien avec le bon serveur, grâce au certificat X.509 présenté, mais également de garantir une confidentialité des échanges, car une fois la connexion établie, le protocole TLS chiffre toutes les données échangées entre le client et le serveur.

5.3. Journalisation

Enfin, la dernière mesure de sécurité prise consiste à historiser l'ensemble des mouvements effectués sur l'application. La journalisation est assurée par un module dédié, logger.php, dont le rôle est de capter les actions significatives effectuées au sein de l'application.



Date / Heure	Action	Message	Utilisateur (session)	IP	Détails (context)
2026-03-24T20:19:59+00:00	Connexion réussie	Connexion réussie (formulaire haché)	alice	172.18.0.1	{"username":"alice"}
2026-03-24T20:19:47+00:00	Inscription	Nouvelle inscription (formulaire haché)	-	172.18.0.1	{"username":"noël.vallée-vaillant","role":"Analyst"}
2026-03-24T20:19:20+00:00	Inscription	Nouvelle inscription (formulaire haché)	-	172.18.0.1	{"username":"nicolas.lenoir","role":"Customer"}
2026-03-24T20:18:23+00:00	Déconnexion	Déconnexion	susan.laroche-mercier	172.18.0.1	-
2026-03-24T20:18:21+00:00	Connexion réussie	Connexion réussie (formulaire haché)	susan.laroche-mercier	172.18.0.1	{"username":"susan.laroche-mercier"}
2026-03-24T20:17:52+00:00	Déconnexion	Déconnexion	susan.laroche-mercier	172.18.0.1	-
2026-03-24T20:17:49+00:00	Modification de solde	Modification du solde d'un compte	susan.laroche-mercier	172.18.0.1	{"account_id":28,"new_balance":"200.00"}

La journalisation joue un rôle clé dans la sécurité de l'application. Elle permet notamment de détecter des tentatives d'accès non autorisées (multiples échecs de connexion), repérer des activités anormales (ex : modifications répétées ou inattendues), enquêter en cas d'incident de sécurité, assurer une forme de traçabilité des actions utilisateurs.

Conclusion

Le projet de création d'une API vulnérable constitue un parcours complet qui permet d'aborder concrètement les enjeux de la sécurité applicative, depuis la conception jusqu'au déploiement de l'application dans un environnement test.

La modélisation de la base de données a posé des fondations solides, car une mauvaise conception initiale peut introduire des failles structurelles durables. L'implémentation volontaire de vulnérabilités — injections SQL, abus de privilèges — a permis d'en mesurer concrètement les conséquences, avant d'y apporter des correctifs efficaces : requêtes préparées, validation des entrées, hachage bcrypt et communications chiffrées via HTTPS et certificat X.509. La journalisation des actions sensibles vient compléter ce dispositif en assurant une supervision continue et une meilleure réactivité en cas d'incident.

Au-delà de la simple correction de failles, ce projet illustre une démarche de sécurité en profondeur où chaque couche — base de données, logique applicative, authentification, transport et supervision — joue un rôle complémentaire et indispensable face aux menaces actuelles.